Core JavaScript Cheatsheet — CITS3403/5505 Ch06 (P1–64, 共64页全覆盖)

A3 Cheatsheet | ★★=核心考点 | 🧠=典型考题 | ⚠=易错陷阱 | 所有代码均有中英注释

1. JS简介 What is JS (P2–5)

- **ECMAScript**标准; 高级/动态/无类型/解释型; **原型式prototype-based**; 函数一等公民
- 三组件: Core JS / Client-side JS / Server-side JS (P2)
- **事件驱动Event-driven**: 用户操作=event, JS=响应事件 (P4)
- 环境: ①浏览器(HTML+browser) ②Node.js(本课不用) (P5)

⚠ 关键验证必须服务器端! 客户端验证可被绕过(P4)

2. 浏览器中JS (P6–9)

引入(P7): ①`onclick=""`内联 ②`<script>`内嵌 ③`<script src="">`外部
调试(P8): DevTools Sources/Console/Breakpoints
IO(P9): `console.log()`调试✓; `alert()`弹窗; `confirm()`→T/F; `prompt()`→字符串/null

⚠ `document.write()`不推荐(P9)

3. 变量&语法 Variables (P11–13)

命名: `$/_`/字母开头, 大小写敏感, camelCase (P11)

```
let z = x+y;       // 块级,可变 ✓默认
const x = 6;       // 块级,不可变 ✓优先
var stop = true;   // 函数级 X避免
zz = 1;            // 全局! X X坏
```

const不能重赋值，但**可修改对象属性**: `const a={x:1};a.x=2;//✓ a={y:3};//X TypeError`
保留字(P12): break/case/class/const/for/function/if/let/return/var/while等
语法(P13): `//`单行 `/* */`多行; `;`结尾, ASI自动补分号

⚠ ASI陷阱: `return`后换行→`return;`→返回undefined! (P13)

4. 类型 Types (P14,P21–22)

原始Primitives: Number,String,Boolean,null,undefined (P14)
对象Objects: Function,Array,Date,Math; 用、访问; 无引用自动GC (P14)
内存: 原始→值在栈; 对象→栈存引用,堆存数据 (P14 Figure 4.1)
null=故意空值(→0) vs **undefined**=未声明/未赋值(→NaN) (P21)
动态类型(P22): 同一变量可改类型; `typeof`返回类型字符串

`typeof 42`→"number"	`typeof {}`→"object"	`typeof null`→"object"⚠
`typeof "s"`→"string"	`typeof fn`→"function"	`typeof []`→"object"(数组也是!)
`typeof true`→"boolean"	`typeof undef`→"undefined"	

5. Number数值 (P15–17)

全为**64位双精度浮点IEEE754**: `0.1+0.2≠0.3` (P15)
运算: `+ - * / % ++ -- += -= *= /= %=`
常量(P16): `MAX/MIN_VALUE NaN ±INFINITY PI`

```
1/0=Infinity; Math.sin(3.5); parseInt("124")=124
parseInt("3.99")=3(截断!); parseInt("hello")=NaN
```

NaN特性(P17)

```
NaN+5=NaN(传染!); NaN===NaN+false(不等于自己!)
isNaN(NaN)=true(用这个检测)
```

6. String字符串 (P18–19)

Unicode16位; 单双引号无区别; 转义`\' \" \\ \n \t`; 无char类型 (P18)

方法(P19)	用途	方法	用途
`charAt(n)`	第n位字符	`indexOf(s)`	子串位置
`substring(a,b)`	截取[a,b)	`toLowerCase/toUpperCase`	大小写

`.length`+长度; `.toString()`=数转串; `+"123"`=123串转数; `"a"+"b"`=拼接

7. Boolean & null/undefined (P20–21)

`true/false; ! && ||`: **&&和||短路求值** (P20)

```
0&&"hi"=0(falsy直接返回); 1&&"hi"="hi"(truthy返回第二个)
""||"def"="def"; "hi"||"def"="hi"
```

	null(P21)	undefined(P21)
含义	故意留空	未声明/未赋值
typeof	"object"⚠	"undefined"
→Number	0	NaN

8. 隐式类型转换 ★★ (P23–26)

→Number(P23)

true→1 false→0; String→数值/NaN; **null→0; undefined→NaN**
例: `4-true+3 7*"3"=21 4+null=4 4+undefined=NaN`

→String(P23)

Number→"值" Bool→"true"/"false" null→"null" undefined→"undefined"

→Boolean ★★(P24)

Falsy(仅5+1个): `false 0 "" undefined NaN null`
Truthy: 其余一切 **包括"0"和"false"和[]和()!**

```
// 实用(P24):
var p=object&&object.getProp(); //安全访问
let n=otherName||"default";     //默认值
Number()/String()/Boolean()     //手动转换
```

== vs === ★★(P25)

```
"dog"=="dog"=T  1==true=T!   'abc'==['abc']=T!!
"dog"!=="dog"=T 1!==true=F   'abc'!==['abc']=F
```

⚠ **永远用===!** ==类型转换→反直觉结果 (P25)

经典陷阱(P26)

```
2+"1"=1(数字) 2+"1"="21"(拼接!) + +号用义性!
"b"+"a"+ +"a"+"a"="baNaNa" + +"a"=NaN
 ''==0=T  0==''0'=T  ''==''0'=F + ==非传递!
```

🧠 **考题:** `"5"+3="53" "5"-3=2 true+true=2 Boolean("0")=true! typeof NaN="number"!`

9. 控制结构 (P27–28)

```
for(let i=0;i<n;i++){...}        //for(P27)
for(let x in obj){obj[x];}       //for-in遍历属性(P27)
while(cond){...}                   //while(P27)
if(a){..}else if(b){..}else{..}  //条件(P28)
switch(v){case 1:..break;default:..} //switch(P28)
let r=cond ? valT : valF;        //三元(P28)
```

⚠ switch忘写break→穿透执行下个case! (P28)
⚠ for...in遍历的是key(字符串索引),不是value!

10. 数组 Arrays (P29–34)

索引从0; length=最高索引+1; 可混合类型; 大小可变 (P30)

```
Array(10)=空len=10; Array()=空len=0 //⚠行为不一致(P31)
Array(10,2,"hi")=[10,2,"hi"]
a[99]="x"→len自动10000; a[50]=undefined不报错! (P32)
```

方法★(P33)

`push/pop`	末尾加/删	`unshift/shift`	头部加/删
`join`	→字符串	`reverse`	反转⚠改原
`sort(fn?)`	排序⚠改原	`concat`	连接→新
`slice(a,b)`	截取→新	`splice(i,n,..)`	插入⚠改原
`delete a[i]`	→undef	length不变!	

⚠ `sort()`默认按字符串! [10,9,80,3].sort()→[10,3,80,9]
正确: `.sort((a,b)=>a-b)`升序 `.sort((a,b)=>b-a)`降序
关联数组(P34): 字符串索引=Object, 无push/pop, .length=0

11. 函数 Functions ★ (P35–42)

`function name(p1,p2){return r;} //须先定义(P36)`

Call by Value vs Sharing ★★(P37)

原始: Call by Value 传副本, 修改不影响外部
对象: Call by Sharing 传引用副本, push等修改原对象✓, 重赋值不影响X

```
function run(x){x+=1;return x;}
let u=1,v=run(u); // u=1,v=21(值传递)

function setOut(p1){
  p1.push("Ivan");  // ✓修改原数组!
  p1=newArr;        // X重赋值不影响外部!
}
```

🧠 `function f(obj,n){obj.v=99;n=99;} let o={v:1},n=1;f(o,n); → o.v=99, n=1`

参数(P38–39)

无类型; 少传→undefined; 多传→arguments获取全部; 形参formal vs 实参actual

```
function findMax(){let m=-Infinity;
  for(let i=0;i<arguments.length;i++)
    if(arguments[i]>m) m=arguments[i];
  return m;}
findMax(4,5,6); //6 (P39)
```

一等公民(P40) / 匿名&箭头(P41) / 递归(P42)

```
let fn=announce;   //不加()=赋值函数本身
let msg=announce();//加()=调用,赋值返回值
String(fn);        //→函数源代码文本(P40)

arr.sort(function(a,b){return b-a;}); //匿名(P41)
arr.sort((a,b)=>{return b-a;});       //箭头(P41)
//sort: 负<a前, 正=b前, 0=不变

var ninja=(yell:function cry(n){ //命名匿名递归(P42)
  return n>0 ? cry(n-1)+"a":"hiy";});
ninja.yell(5); //"hiyaaaaa"
```

12. 对象 Objects (P43–49)

对象=名值对(名String,值任意)=HashMap/Dict (P44); 非原始都是对象(数组函数也是!)

```
let tea={ingredients:["Tea"],taste:"Good",
  time:function(){return 4;}}; //(P44)
obj.name; obj["name"]; //两种访问(P45)
obj.x=1; delete obj.x; for(let k in obj){} //增删遍历(P46)
```

Global(P47): `window/globalThis`; 全局属性无需前缀

this关键字(P48)

方法中→所属对象	单独/函数→全局	严格模式→undefined
事件中→触发元素	call()/apply()→任意对象	

构造函数+new(P49)

```
function Drink(p){this.price=p;
  this.get=function(){return this.price*1.125;}}
let d=new Drink(10); //⚠每次new创新函数→浪费!
```

13. 原型 Prototype ★★ (P50–54)

JS不用class→**原型式prototype-based**编程(P50); 无类/面向原型/基于实例

方法放prototype(最佳)(P51)

```
function Drink(p){this.price=p;} //数据在实例
Drink.prototype.get=function(){ //方法在prototype
  return this.price*1.125;
} //所有实例共享同一函数!
```

原型链★★(P52)

查找: 实例→`Drink.prototype→Object.prototype→null`
prototype被所有实例共享, 赋值给prototype→所有实例通过this可用, 根=Object.prototype

运行时扩展(P53)

```
String.prototype.reversed=function(){
  let r="";for(let i=this.length-1;i>=0;i--)r+=this[i];return r;}
"live on".reversed(); //"no evil" +所有String立刻可用!
```

可用prototype实现继承: 子类prototype=父类实例(P53)

new三步★★(P54)

①创新对象proto=X.prototype ②调构造this=新对象 ③返回对象(除非显式return另一对象)

🧠 **原型链shadow:**

```
function Cat(n){this.name=n;}
Cat.prototype.speak=function(){return "Meow";}
let c=new Cat("M");
c.speak=function(){return "woof!";}//实例属性
c.speak();        //"woof!"=实例遮蔽prototype
delete c.speak;   //删实例属性
c.speak();        //"Meow"=回落到prototype!
```

14. 作用域 Scoping ★★ (P55–61)

三种(P56): Global(整文档) / Function(函数内) / Block({...}内); 内层可见外层(套娃)
善恶(P57): 🟩const(纯善)→ 🟨let(善)→ 🟥var(恶)→ 🟥undeclared(纯恶)

四种声明对比★★(P58–61)

声明	作用域	可变	加window	#块外	函数外
const	**Block**	✗	✗	✗	✗
let	**Block**	✓	✗	✗	✗
var	**Function**	✓	✓	✓泄漏	✗
未声明	**Global!**	✓	✓	✓	✓到处

const/let→块外不可见→安全(P58); var→泄漏到函数级→可替代(P59); 未声明→全局泄漏(P60)
函数外声明(无论哪种)都可在函数内访问; 只**有var加到window上**(P61)

```
function t(){if(true){var a=1;let b=2;} console.log(a);//1 console.log(b);//Error!}
```

★★经典for循环陷阱

```
var fns=[]; for(var i=0;i<3;i++){fns.push(function(){return i;})}
fns[0]();fns[1]();fns[2](); //3,3,3←var共享一个i!

let fns2=[];for(let j=0;j<3;j++){fns2.push(function(){return j;})}
fns2[0]();fns2[1]();fns2[2](); //0,1,2←let每次独立!
```

15. 闭包 Closures ★★ (P62–64)

内部函数可访问外层变量(P62); **闭包=函数的局部变量在函数返回后仍存活**(P62)

经典makeAdder(P63)

```
function makeAdder(a){          //a被闭包捕获
  return function(b){return a+b;} //记住a
}
let add5=makeAdder(5),add20=makeAdder(20);
add5(6)=11; add20(7)=27; //互不干扰!
```

机制(P63–64)

- 每次执行→新scope object保存局部变量(P63)
- 返回的函数持有scope引用→不被GC(P64)
- 每次调用→新scope→互不干扰(P64)
- vs全局: ①每次新scope ②不可直接访问(P64)

终极题: var+setTimeout

```
for(var i=0;i<3;i++){setTimeout(function(){console.log(i);},100);}
//输出3,3,3←var一个i,循环结束=3
//修复:var→let→每次独立→输出0,1,2
```

实用:私有变量

```
function makeCounter(){
  let count=0; //私有!
  return {inc:function(){return ++count;},
          get:function(){return count;}};
}
let c=makeCounter();c.inc();//1 c.inc();//2 c.get();//2
```

16. 易混淆知识点: 关键判断

对比	A	B	判断
==vs===	类型转换后比	类型+值都比	**永远===**
null vs undef	故意空→0	未赋值→NaN	typeof null="object"⚠
let vs var	块级	函数级	var泄漏出块!
push vs 赋值	改原对象✓	不影响原参✗	call by sharing
slice vs splice	不改原	改原	splice=破坏
sort默认	按字符串	传(a,b)=>a-b	数字必传比较器
构造内vs proto	每实例一份	共享一份	proto节省内存
+号两义	有Str→拼接	无Str→数学	-*/%数学

17. 速查表

Falsy完整(仅6个)

`false 0 "" null undefined NaN` — 其余一切truthy(包括"0" "false" [] {})

+号行为

```
Num+Num=加法(3+4=7); Str+任何→拼接("3"+4="34")
Num+Bool=加法(3+true=4); -*/% →全转Num=数学
```

常见转换

`Number("")=0`	`Number("0")=0`	`Number(true)=1`
`Number(null)=0`	`Number(undefined)=NaN`	`Number("abc")=NaN`
`Boolean("")=F`	`Boolean("0")=T!`	`Boolean(0)=F`
`Boolean([])=T!`	`Boolean({})=T!`	`Boolean(null)=F`

18. 综合考题精选

Q1 const对象: `const a=[1,2];a.push(3);//✓ a=[4,5];//✗TypeError`

Q2 for...in数组: `for(let x in [10,20]){console.log(x,typeof x);}` → "0" string, "1" string ←索引的字符串!

Q3 prototype共享:

```
function A(x){this.x=x;this.f=function(){return this.x}}
function B(x){this.x=x;} B.prototype.f=function(){return this.x;}
new A(1).f===new A(2).f; //false(每次新函数)
new B(1).f===new B(2).f; //true(prototype共享)
```

Q4 ASI: `function f(){return\n{name:"A"}} f()→undefined!` (return后换行→ASI→return;)

Q5 闭包计数: `function m(){let n=0;return function(){return ++n;}} let c1=m(),c2=m(); c1();c1();c1()→1,2,3; c2()→1(独立scope!)`

Q6 +号陷阱: `"5"+3+1="531"`(一旦Str一直拼) vs `"5"-3+1=3`(减法做数学→2+1=3)

Q7 null===undefined: `null==undefined→true null===undefined→false`(类型不同)

Q8 this指向:

```
let o={name:"A",greet:function(){return this.name;}};
o.greet();          //"A" ←this=o(方法的所属对象)
let f=o.greet;
f();                //undefined ←this=window(函数调用)
```

Q9 Array越界: `let a=[1,2];a[10]=99; a.length→11; a[5]=undefined`(不报错!)

Q10 typeof陷阱: `typeof null="object"(bug!) typeof NaN="number"(NaN是number!) typeof []="object"(数组是对象!)`

19. 更多典型考题与代码示例

Q11 完整的构造函数+原型示例

```
// 构造函数定义数据(P49)
function Student(name, id) {
  this.name = name;      // 实例属性: 每个学生不同
  this.id = id;
}
// 方法放prototype(P51): 所有Student共享
Student.prototype.greet = function() {
  return "Hi, I'm " + this.name + " (#" + this.id + ")";
};
Student.prototype.study = function(subject) {
  return this.name + " is studying " + subject;
};

let s1 = new Student("Alice", 1001);  // new的三步(P54):
let s2 = new Student("Bob", 1002);    // ①创对象 ②绑this ③返回
console.log(s1.greet());      // "Hi, I'm Alice (#1001)"
console.log(s2.study("JS")); // "Bob is studying JS"
s1.greet === s2.greet;        // true = prototype共享!
```

Q12 闭包: 创建多个独立计时器

```
function createTimer(name) {
  let seconds = 0;            // 闭包私有变量
  return {
    tick: function() { seconds++; return name+": "+seconds+"s"; },
    reset: function() { seconds = 0; }
  };
}
let t1 = createTimer("Timer1");  // 独立scope: {name:"Timer1",seconds:0}
let t2 = createTimer("Timer2");  // 独立scope: {name:"Timer2",seconds:0}
t1.tick(); // "Timer1: 1s"
t1.tick(); // "Timer1: 2s"
t2.tick(); // "Timer2: 1s" ← 互不影响!
t1.reset(); t1.tick(); // "Timer1: 1s" ← 重置后重新开始
```

Q13 综合: 类型转换链式推导

```
// 逐步求值(从左到右):
"" + 1 + 0     → "1" + 0 → "10"      // ""是Str→一直拼接
"" - 1 + 0     → -1 + 0 → -1         // -只做数学
true + false   → 1 + 0 → 1           // Bool→Num
"2" * "3"      → 6                   // *只做数学
4 + 5 + "px"   → 9 + "px" → "9px"    // 先数字后拼接
"$" + 4 + 5    → "$4" + 5 → "$45"    // 先拼接一直拼接
"4" - 2        → 2                   // -只做数学
"4px" - 2      → NaN                 // "4px"→NaN
7 / 0          → Infinity            // 除以0→正无穷
typeof typeof 1 → "string"           // typeof 1="number"(字符串), typeof "number"="string"
```

Q14 作用域综合题

```
let x = 10;
function outer() {
  let x = 20;           // 遮蔽外层x
  function inner() {
    let x = 30;         // 再遮蔽
    console.log(x);     // 30 ← 最近的x
  }
  inner();
  console.log(x);       // 20 ← outer的x
}
outer();
console.log(x);         // 10 ← 全局的x
```

Q15 对象属性动态计算

```
let key = "name";
let obj = {};
obj[key]= "Alice";      // 等价于 obj.name = "Alice"
obj["age"] = 20;
for(let k in obj) {
  console.log(k + ":" + obj[k]); // "name:Alice" "age:20"
}
delete obj.age;
console.log(obj.age);   // undefined (已删除)
```

20. 常见代码模式速查

安全属性访问(P24)

```
let val = obj && obj.prop && obj.prop.sub;
// 如果obj是null→短路→不报错→val=null
```

默认参数值(P24+P38)

```
function greet(name) {
  name = name || "World"; // 未传参→undefined→falsy→用默认
  return "Hello, " + name;
}
```

IIFE立即执行函数

```
(function() {
  let private = "secret"; // 不污染全局
  console.log(private);
})(); // 立即执行
```

数组操作链

```
let arr = [3,1,4,1,5,9];
arr.sort((a,b)=>a-b)    // [1,1,3,4,5,9] 排序
   .reverse()           // [9,5,4,3,1,1] 反转
   .slice(0,3);         // [9,5,4]       取前3个
```

for...in vs for循环(遍历数组)

```
// ✓ 推荐: 传统for
for(let i=0;i<arr.length;i++){arr[i];}
// ⚠ for...in得到的是字符串索引"0","1"
for(let i in arr){typeof i;} // "string"!
```

检测类型

```
typeof x === "number"    // 检测数字
typeof x === "string"    // 检测字符串
typeof x === "function"  // 检测函数
Array.isArray(x)         // 检测数组(typeof不行!)
x === null               // 检测null(typeof返回"object"!)
```

21. 完整HTML+JS示例(P4,P7,P9,P36)

```
<!DOCTYPE html>
<html>
<head>
<script>
function validate() {                  // P36:定义函数
  let name = document.getElementById("name").value;
  if (name === "") {                   // P25:===严格比较
    alert("Name required!");           // P9:alert输出
    return false;                      // 阻止表单提交
  }
  return true;
}
</script>
</head>
<body>
<form onsubmit="return validate()">  <!-- P4:事件驱动 -->
  <input id="name" type="text">
  <button type="submit">Submit</button>
</form>
<p id="demo"></p>
<button onclick="document.getElementById('demo').innerHTML='Changed!'">
  Click me                             <!-- P7:onclick内联 -->
</button>
</body>
</html>
```

22. 原型链可视化(P52)

```
┌──────────────┐
│ coffee       │ ← new Drink("Coffee",5)
│ name:"Coffee"│ ← 实例自身属性
│ price: 5     │
└──────────────┘
       │ __proto__
       ▼
┌──────────────┐
│ Drink.prototype │ ← 所有Drink实例共享
│ getPrice:function │
│ describe:function │
└──────────────┘
       │ __proto__
       ▼
┌──────────────┐
│ Object.prototype │ ← 原型链的根
│ toString:function │
│ hasOwnProperty:fn │
└──────────────┘
       │ __proto__
       ▼
     null            ← 链的终点
```

查找 `coffee.getPrice()`: 自身没有→在Drink.prototype找到✓
查找 `coffee.toString()`: 自身没有→Drink.prototype没有→Object.prototype找到✓
查找 `coffee.xyz`: 全部没有→返回undefined